

SQL LAB

Experiments 13 – 27

*Unit 6 · DML / DQL Lab Sheet***SCHEMA — CREATE ONCE, USE FOR ALL EXPERIMENTS**

```
CREATE DATABASE lab_unit6;
USE lab_unit6;

CREATE TABLE Class (
    class_id INT PRIMARY KEY,
    class_name VARCHAR(50) NOT NULL,
    section VARCHAR(10)
);

CREATE TABLE Student (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL UNIQUE,
    address VARCHAR(100) DEFAULT 'Birtamode',
    class_id INT NOT NULL,
    section VARCHAR(50),
    age INT DEFAULT 16 CHECK (age >= 15),
    FOREIGN KEY (class_id) REFERENCES Class(class_id)
);
```

LAB EXPERIMENTS**EXPERIMENT 13: INSERT Statement**

Write SQL queries to perform the following *INSERT* operations on the *Student* table:

- Insert a single student record.
- Insert multiple student records (at least 3) in a single query.
- Insert a student record using the `INSERT INTO ... SET` syntax.
- Insert a student record where address uses the `DEFAULT` value.
- Create a backup table with the same structure and copy students older than a specific age into it using `INSERT INTO ... SELECT`.

EXPERIMENT 14: SELECT Statement (Basic Retrieval & FROM Clause)

Write SQL queries to perform the following SELECT operations on the Student table:

- a) Display all columns and all rows.
- b) Display only the name and age of all students.
- c) Display all unique addresses from the Student table.
- d) Display name as 'Student Name' and age as 'Student Age' for all students.
- e) Display only the first 3 students from the Student table.

EXPERIMENT 15: WHERE Clause (Filtering & Operators)

Write SQL queries to perform the following filtering operations on the Student table:

- a) Find students whose age is greater than 20.
- b) Find students who live in 'Kathmandu' AND are older than 18.
- c) Find students who live in 'Kathmandu' OR 'Pokhara'.
- d) Find students whose age is between 18 and 22.
- e) Find students whose class_id is 4, 5, or 6.
- f) Find students whose name starts with 'R'.
- g) Find students whose name ends with 'a'.
- h) Find students whose address is NULL.

EXPERIMENT 16: ORDER BY Clause (Sorting)

Write SQL queries to perform the following sorting operations on the Student table:

- a) Select all students and sort by student_id in descending order.
- b) Select name and age of all students, sorted by age in ascending order.
- c) Select name and age of all students, sorted by age in descending order.
- d) Select all students sorted by class_id ASC, and within class, sort by name alphabetically.
- e) Select all students whose age is greater than 20, sorted by name alphabetically.

EXPERIMENT 17: Aggregate Functions

Write SQL queries to perform the following aggregate operations on the Student table:

- a) Count total number of students.
- b) Count number of students whose name ends with 'a'.
- c) Find sum of ages of all students.
- d) Find average age of all students.
- e) Find average age of students whose class_id is 4.
- f) Find maximum age among all students.
- g) Find minimum age among students living in 'Kathmandu'.
- h) Count number of unique addresses in the Student table.

EXPERIMENT 18: GROUP BY Clause

Write SQL queries to perform the following grouping operations on the Student table:

- a) Count number of students in each class_id. Display class_id and count.
- b) Find total age of students in each class_id. Display class_id and total age.
- c) Find average age of students in each class_id. Display class_id and average age.
- d) Count students in each class_id and address combination. Display all 3 columns.
- e) Count students in each class_id who live in 'Kathmandu'. Display class_id and count.

EXPERIMENT 19: HAVING Clause

Write SQL queries to perform filtering on grouped data using HAVING:

- a) Find class_ids that have more than 2 students. Display class_id and count.
- b) Find class_ids where total age of students is > 50. Display class_id and total age.
- c) Find class_ids where average age is > 20. Display class_id and average age.
- d) Find class_ids that have more than 2 students living in 'Kathmandu'.

EXPERIMENT 20: INNER JOIN

Create a Class table with appropriate columns and establish a relationship with Student table. Write SQL queries to perform the following JOIN operations:

- a) Display student name and class name for all students.
- b) Display student name, age, class name, and section for all students.
- c) Display student name, age, class name, and section for students older than 20.
- d) Display class name and the number of students in each class.

EXPERIMENT 21: LEFT JOIN / RIGHT JOIN

Using Student and Class tables, insert one student with no class (class_id = NULL) and one class with no students. Write SQL queries for:

- Display all students and class names using LEFT JOIN (include unassigned students).
- Display all classes and students using RIGHT JOIN (include empty classes).
- Display all students and class names using LEFT JOIN for students older than 18.

EXPERIMENT 22: CROSS JOIN

Using Student and Class tables, write SQL queries to perform the following CROSS JOIN operations:

- Display all possible combinations of student names and class names using CROSS JOIN.
- Display all possible combinations where student age is greater than 18.

EXPERIMENT 23: Non-Correlated Subqueries

Using Student and Class tables, write SQL queries using subqueries:

- Find student names whose class_id is in classes having >2 students using IN.
- Find students whose age is greater than average student age.
- Find students in the same class as 'Ramesh Sharma'.
- Find names of students who have a class assigned using EXISTS.

EXPERIMENT 24: Correlated Subqueries

Using Student and Class tables, write SQL queries using correlated subqueries:

- Find names of students whose age is > average age of students in their own class_id.
- Find students having a class_id that exists in Class table using correlated EXISTS.

EXPERIMENT 25: UPDATE Statement

Using Student and Class tables, write SQL queries to perform UPDATE operations:

- Update age of a specific student (by student_id) to a new value.
- Increase age of all students in a specific class_id by 1.
- Update address of students to a specific city if their class name matches a value.
- Update class_id of students to a specific value if older than a specific age.

EXPERIMENT 26: DELETE Statement

Using Student and Class tables, write SQL queries to perform DELETE operations:

- a) Delete a specific student (by student_id).
- b) Delete all students whose age is greater than a specific value.
- c) Delete all students who belong to a specific class name using a subquery.

EXPERIMENT 27: Views (CREATE, ALTER, DROP)

Using Student and Class tables, write SQL queries to perform VIEW operations:

- a) Create a view displaying student_id, name, and age of all students.
- b) Create a view displaying student name and class name using JOIN.
- c) Create a view displaying class_id and number of students in each class.
- d) Query the JOIN view to display only students older than 18.
- e) Drop one of the created views.